

Chapter 40: Prefix Sums

Personal Notes

Imagine you're asked: "What's the sum of `arr[3..7]`?" — over and over, for a thousand different ranges. Recomputing each time is $O(n)$ per query. That's fine once, but wasteful for a thousand queries.

Prefix sums solve this with a beautiful idea: PRE-COMPUTE the running total once, then answer any range-sum query in $O(1)$. It's one of the simplest yet most powerful techniques in DSA — often the missing piece that turns $O(n^2)$ subarray problems into $O(n)$. And when paired with a `HashMap`, prefix sums unlock a whole family of "subarrays that sum to `K`" problems that sliding window can't touch (because of negative numbers).

1. The Core Idea

A PREFIX SUM at index `i` is the sum of all elements from 0 to `i` (inclusive). Store these in an array — then any RANGE SUM is just a subtraction.

```
arr:      [3, 1, 4, 1, 5, 9, 2]
index:    0  1  2  3  4  5  6

prefix:   [3, 4, 8, 9, 14, 23, 25]

prefix[0] = 3           (just arr[0])
prefix[1] = 3 + 1 = 4
prefix[2] = 3 + 1 + 4 = 8
prefix[6] = total = 25

Sum from index 2 to 5:
  = prefix[5] - prefix[1]
  = 23 - 4 = 19
  = (arr[2] + arr[3] + arr[4] + arr[5])
  = 4 + 1 + 5 + 9 = 19 ✓
```

The magic formula: `sum(l..r) = prefix[r] - prefix[l-1]`. Any range sum in $O(1)$ after building the prefix array once in $O(n)$. That trade-off — pay $O(n)$ once, get $O(1)$ forever — is the whole trick.

2. Building the Prefix Sum Array

Two common styles. Both work — the difference is just how you handle boundaries.

2.1 Style A: Same Size, Direct Prefix

```
int[] arr = {3, 1, 4, 1, 5, 9, 2};
```

```

int n = arr.length;
int[] prefix = new int[n];

prefix[0] = arr[0];
for (int i = 1; i < n; i++) {
    prefix[i] = prefix[i - 1] + arr[i];
}

// sum(l..r):
int sum = (l == 0) ? prefix[r] : prefix[r] - prefix[l - 1];

```

2.2 Style B: Size n+1 with Leading 0 (Cleaner)

```

int[] arr = {3, 1, 4, 1, 5, 9, 2};
int n = arr.length;
int[] prefix = new int[n + 1];    // prefix[0] = 0

for (int i = 0; i < n; i++) {
    prefix[i + 1] = prefix[i] + arr[i];
}

// sum(l..r):    NO SPECIAL CASE
int sum = prefix[r + 1] - prefix[l];

```

Prefer style B for interviews: The size-(n+1) version with a leading 0 removes the `l==0` special case — cleaner code, fewer bugs. Think of `prefix[i]` as "sum of the first *i* elements," not "sum up to index *i*." That mental shift makes the formula `prefix[r+1] - prefix[l]` feel natural.

3. Complexity Comparison

Operation	Time	Space
Build prefix array	$O(n)$ — once	$O(n)$
Single range sum (with prefix)	$O(1)$	$O(1)$
Single range sum (brute force)	$O(n)$	$O(1)$
Q queries (with prefix)	$O(n + Q)$	$O(n)$
Q queries (brute force)	$O(n \times Q)$	$O(1)$

For anything more than a handful of queries, prefix sums are a massive win — often the difference between a passing solution and a Time Limit Exceeded.

4. Classic Problem 1 — Range Sum Query (Immutable)

Design a class that answers repeated range-sum queries on a fixed array. LeetCode 303 — the textbook prefix sum problem.

```
class NumArray {
    private int[] prefix;

    public NumArray(int[] nums) {
        int n = nums.length;
        prefix = new int[n + 1];
        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }

    public int sumRange(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
}

// Constructor: O(n)
// Each query: O(1)
```

5. Classic Problem 2 — Subarray Sum Equals K

Given an array (possibly with negatives) and integer K, count how many CONTIGUOUS subarrays sum exactly to K. This is where prefix sums + HashMap shine — sliding window can't handle negatives.

The Key Insight

If $\text{prefix}[j] - \text{prefix}[i] == K$, then the subarray from $i+1$ to j sums to K. Rearrange: $\text{prefix}[i] == \text{prefix}[j] - K$. So for each j , ask: "how many earlier prefix sums equal $\text{prefix}[j] - K$?" A HashMap of prefix-sum FREQUENCIES answers that instantly.

```
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1); // empty prefix = sum 0, occurs once

    int prefix = 0, count = 0;
    for (int num : nums) {
        prefix += num;
        count += prefixCount.getOrDefault(prefix - k, 0);
        prefixCount.merge(prefix, 1, Integer::sum);
    }
    return count;
}
```

```
// Time: O(n) Space: O(n)
```

Dry Run for nums=[1, 1, 1], k=2

```
Initial: prefixCount = {0: 1}, prefix = 0, count = 0

num=1: prefix = 1
      looking for prefix - k = -1 in map → not found
      prefixCount = {0: 1, 1: 1}

num=1: prefix = 2
      looking for prefix - k = 0 → found! (count 1)
      count = 1
      prefixCount = {0: 1, 1: 1, 2: 1}

num=1: prefix = 3
      looking for prefix - k = 1 → found! (count 1)
      count = 2
      prefixCount = {0: 1, 1: 1, 2: 1, 3: 1}

Result: 2 subarrays sum to k=2: [1,1] (indices 0-1) and [1,1]
(indices 1-2)
```

Why sliding window fails here: Sliding window assumes that adding elements monotonically increases the sum, so you can shrink from the left when the sum exceeds the target. With **NEGATIVE** numbers, adding an element might **DECREASE** the sum — breaking the assumption. Prefix sums + HashMap sidesteps this completely.

Why prefixCount.put(0, 1) at the start: This handles subarrays that start at index 0. If a prefix sum equals K exactly, then subtracting K gives 0 — and we need 0 to be counted once as a valid "empty starting point." Forget this line and you'll undercount by exactly the number of subarrays starting at index 0.

6. Classic Problem 3 — Continuous Subarray Sum (Divisible by K)

Given an array and integer K, does any contiguous subarray of length ≥ 2 sum to a multiple of K? A variant of prefix sums using MODULO.

The Insight

Two prefix sums with the **SAME REMAINDER** when divided by K mean the subarray between them sums to a multiple of K. Store (remainder → earliest index) and check if the current remainder was seen far enough back.

```
public boolean checkSubarraySum(int[] nums, int k) {
    Map<Integer, Integer> remainderIndex = new HashMap<>();
    remainderIndex.put(0, -1); // handle subarray starting at 0
```

```

int prefix = 0;
for (int i = 0; i < nums.length; i++) {
    prefix += nums[i];
    int rem = prefix % k;

    if (remainderIndex.containsKey(rem)) {
        if (i - remainderIndex.get(rem) >= 2) return true;
    } else {
        remainderIndex.put(rem, i);    // store EARLIEST index only
    }
}
return false;
}

```

This trick — store remainder → earliest index — appears in many problems: subarrays divisible by K, count of divisible subarrays, longest such subarray.

7. Classic Problem 4 — Maximum Subarray (Kadane vs Prefix)

Find the contiguous subarray with the maximum sum. There are two approaches — both worth knowing.

Approach 1: Prefix Sums

```

public int maxSubArray(int[] nums) {
    int minPrefix = 0;    // smallest prefix seen so far
    int prefix = 0;
    int best = Integer.MIN_VALUE;

    for (int num : nums) {
        prefix += num;
        best = Math.max(best, prefix - minPrefix);
        minPrefix = Math.min(minPrefix, prefix);
    }
    return best;
}

```

For each ending position, the max subarray sum ending here = current prefix - smallest prefix before it. Track the min-prefix as you go.

Approach 2: Kadane's Algorithm

```

public int maxSubArray(int[] nums) {
    int currSum = nums[0], best = nums[0];
    for (int i = 1; i < nums.length; i++) {
        currSum = Math.max(nums[i], currSum + nums[i]);
        best = Math.max(best, currSum);
    }
    return best;
}

```

Kadane's is slightly shorter, but the prefix version generalizes better to related problems (like maximum subarray sum with at least K elements).

8. 2D Prefix Sums — Rectangle Sums

The same idea in 2D. Build a matrix where $\text{prefix}[i][j]$ = sum of the rectangle from (0,0) to (i-1, j-1). Any sub-rectangle sum becomes $O(1)$ — via the inclusion-exclusion formula.

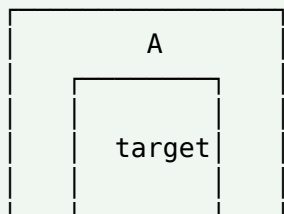
Building the 2D Prefix

```
int[][] matrix = ...; // r x c
int r = matrix.length, c = matrix[0].length;
int[][] prefix = new int[r + 1][c + 1];

for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) {
        prefix[i + 1][j + 1] = matrix[i][j]
            + prefix[i][j + 1]
            + prefix[i + 1][j]
            - prefix[i][j];
    }
}
```

Rectangle Sum in $O(1)$

Sum of rectangle from (r1, c1) to (r2, c2):



A = whole from (0,0) to (r2,c2)
B = strip above the target
C = strip left of the target
D = overlap (top-left corner)

```
target = A - B - C + D
        = prefix[r2+1][c2+1]
          - prefix[r1][c2+1]
          - prefix[r2+1][c1]
          + prefix[r1][c1]
```

```
public int rectangleSum(int r1, int c1, int r2, int c2) {
    return prefix[r2 + 1][c2 + 1]
        - prefix[r1][c2 + 1]
        - prefix[r2 + 1][c1]
        + prefix[r1][c1];
}
```

Why the + prefix[r1][c1]: When you subtract both B and C, you subtract the top-left overlap (D) TWICE. Adding it back once corrects for the double-subtraction. This inclusion-exclusion trick appears in many 2D problems — worth memorizing the pattern.

9. Difference Arrays — The Inverse Trick

If prefix sums turn range-sum queries into $O(1)$, DIFFERENCE ARRAYS do the reverse: turn range-UPDATES into $O(1)$. Perfect when you have many "add X to arr[l..r]" operations and only need the final array at the end.

The Idea

- Create an array `diff[]` of size `n+1`, all zeros
- For each range update (`l, r, delta`): `diff[l] += delta; diff[r+1] -= delta`
- After all updates, take the PREFIX SUM of `diff[]` — that's the final array

```
int[] applyUpdates(int n, int[][] updates) {
    int[] diff = new int[n + 1];

    for (int[] u : updates) {
        int l = u[0], r = u[1], delta = u[2];
        diff[l] += delta;
        if (r + 1 <= n) diff[r + 1] -= delta;
    }

    int[] result = new int[n];
    int running = 0;
    for (int i = 0; i < n; i++) {
        running += diff[i];
        result[i] = running;
    }
    return result;
}

// U updates in O(U), then O(n) once to reconstruct. Total O(U + n).
// Naive approach: O(U × n).
```

Example

`n = 5, updates = [(1, 3, +2), (0, 2, +1), (2, 4, +3)]`

Naive: for each update, loop through the range → $O(U \times n)$

Diff: start `diff = [0, 0, 0, 0, 0, 0]`
`(1, 3, +2): diff = [0, 2, 0, 0, -2, 0]`
`(0, 2, +1): diff = [1, 2, 0, -1, -2, 0]`
`(2, 4, +3): diff = [1, 2, 3, -1, -2, -3]`

Prefix sum: [1, 3, 6, 5, 3] ← final answer

10. When to Reach for Prefix Sums

If you need to...	Use
Repeated range sum queries	1D prefix sum
Repeated 2D rectangle sums	2D prefix sum
Count subarrays with sum = K (any array)	Prefix + HashMap
Longest subarray with sum = K	Prefix + HashMap (earliest index)
Subarray sum divisible by K	Prefix modulo + HashMap
Subarrays with equal count of 0s and 1s	Prefix + HashMap (map 0 → -1)
Range update, query at end	Difference array
Maximum subarray sum (with negatives)	Prefix + track min-prefix, or Kadane
Product of subarrays (positives only)	Prefix products

11. Prefix Sums vs Sliding Window

Aspect	Prefix Sums	Sliding Window
Signal words	"Sum equals K" / "divisible by" / range queries	"Longest/shortest with property"
Handles negatives	Yes — HashMap doesn't care about signs	No — assumes monotonic sum
Handles repeated queries	Yes — $O(1)$ per query after prep	No — one traversal only
Setup cost	$O(n)$ to build the array	None — start scanning immediately
Space	$O(n)$ for prefix array or map	$O(1)$ typically
Typical combo	Prefix + HashMap for count/existence	Two pointers + running state

The dividing rule: If the array has ONLY positive numbers AND you want longest/shortest with a target — sliding window wins. If the array has negatives OR you need count/existence of exact sums — prefix + HashMap. That single distinction picks the right tool 95% of the time.

12. Common Mistakes

Mistake 1: Forgetting `prefixCount.put(0, 1)` at initialization

This entry represents "before we started, we had a prefix sum of 0." Without it, you miss every subarray that starts at index 0. Result: count is too low by the number of prefixes equal to K.

Mistake 2: Using `==` to compare with target (integer overflow)

For very large arrays with big values, prefix sums can overflow `int`. Switch to `long` for both the prefix variable and the `HashMap` keys when values might be large.

Mistake 3: Off-by-one in range boundaries

```
// Style B (n+1 sized array):
sum(l..r) = prefix[r + 1] - prefix[l];    // ✓ inclusive on both ends

// Common mistake:
sum(l..r) = prefix[r] - prefix[l];        // ✗ misses arr[r]
```

Mistake 4: Using prefix + sliding window together carelessly

They're two different tools. Don't mix them. If your array has negatives, use prefix + `HashMap`. If it doesn't, use sliding window. Choose one strategy up front.

Mistake 5: Wrong modulo handling in negatives

```
// In Java, -7 % 3 = -1 (not 2 like in math)
int rem = prefix % k;
if (rem < 0) rem += k;    // normalize to be non-negative

// Otherwise your remainder-index map key comparison silently misses
// matches.
```

Mistake 6: Recomputing prefix inside a query loop

The whole point is to build it ONCE. If you're rebuilding the prefix array for every query, you've gained nothing over brute force. Build outside the query loop.

Mistake 7: Storing latest index in modulo problems instead of earliest

For "longest subarray with sum divisible by K," you want the EARLIEST occurrence of each remainder — because that gives the biggest distance. Update the `HashMap` only if the key isn't already there.

13. Best Practices

- Prefer the size-(n+1) style — it removes edge-case checks
- Use `long` instead of `int` when values might overflow
- For sum-related `HashMap` problems, always initialize with (0, 1) or (0, -1) as appropriate

- For "count subarrays" problems, prefix + HashMap of FREQUENCIES
- For "longest subarray" problems, prefix + HashMap of EARLIEST INDEX
- For modulo problems, normalize negative remainders: $(x \% k + k) \% k$
- For range updates without queries in between, use a difference array
- Draw the prefix array on paper for small inputs before coding — reveals off-by-ones fast

14. Quick Reference

Concept	Meaning
Prefix Sum	Running total: $\text{prefix}[i] = \text{arr}[0] + \dots + \text{arr}[i]$
Range sum formula	$\text{sum}(l..r) = \text{prefix}[r+1] - \text{prefix}[l]$ (n+1 style)
Preferred style	Size n+1 with $\text{prefix}[0] = 0$ as a safety base
Prefix + HashMap	Combo for count/exists of subarrays with target sum
Initial map entry	$\text{put}(0, 1)$ for counting; $\text{put}(0, -1)$ for longest
Frequency map	For "how many subarrays sum to K"
Earliest-index map	For "longest subarray with sum property"
Modulo variant	Remainders instead of raw sums (divisibility problems)
2D prefix sum	Inclusion-exclusion for rectangle sums
Difference array	Turn range updates into $O(1)$; query with a final prefix
Sliding window contrast	Use SW for positives only; prefix for anything else

15. Practice Problems

Try in order — first the basic range-sum patterns, then the HashMap combos.

Foundational

1. Range Sum Query — Immutable (LeetCode 303).
2. Running Sum of 1D Array (LeetCode 1480).
3. Find Pivot Index (LeetCode 724).
4. Left-and-Right Sum Differences (LeetCode 2574).

Subarray Sum Family

5. Subarray Sum Equals K (LeetCode 560) — the classic prefix + HashMap.

6. Contiguous Array (LeetCode 525) — equal 0s and 1s (map 0 → -1).
7. Subarray Sums Divisible by K (LeetCode 974).
8. Continuous Subarray Sum (LeetCode 523) — divisible by K, length ≥ 2 .
9. Binary Subarrays With Sum (LeetCode 930).
10. Number of Submatrices That Sum to Target (LeetCode 1074) — 2D variant.

Max / Min Subarray

11. Maximum Subarray (LeetCode 53) — Kadane or prefix.
12. Maximum Sum Circular Subarray (LeetCode 918).
13. Maximum Length of Repeated Subarray (LeetCode 718).
14. Longest Well-Performing Interval (LeetCode 1124).

2D Prefix Sum

15. Range Sum Query 2D — Immutable (LeetCode 304).
16. Maximum Sum of Rectangle No Larger Than K (LeetCode 363).

Difference Array

17. Corporate Flight Bookings (LeetCode 1109) — range updates.
18. Car Pooling (LeetCode 1094).
19. Range Addition (LeetCode 370).

16. Final Takeaway

Prefix sums are one of those "why didn't I think of that?" techniques. Once the mental switch clicks — pay $O(n)$ once, get $O(1)$ forever — you'll see prefix sum opportunities in dozens of problems that used to look hopeless.

For interviews, master three patterns: the pure prefix array for range queries, prefix + HashMap for subarray-sum-equals-K problems, and 2D prefix sums for rectangle queries. Those three combined with the difference array trick handle nearly every prefix-sum interview question you'll see.

Key Takeaway: Prefix sum = running total; range sum = subtraction. Use $n+1$ -sized array with a leading 0 for clean formulas. Pair with HashMap for subarray sums with negatives (where sliding window fails). Initialize map with (0, 1) for count problems or (0, -1) for longest-subarray problems. Difference array is the inverse trick for fast range updates. Normalize negative remainders in modulo problems.

17. What's Next?

With prefix sums covered, the DSA toolkit is nearly complete. Remaining specialized topics:

- Two Pointers in depth — palindrome, 3-sum, container with most water
- Greedy Algorithms — activity selection, interval scheduling, huffman coding
- Segment Tree / Fenwick Tree — range queries WITH updates
- Advanced Strings — KMP, Rabin-Karp for pattern matching
- Advanced Graphs — Bellman-Ford, Floyd-Warshall, articulation points
- Number Theory basics — GCD tricks, primes, modular arithmetic
- System Design — the next big step in interview prep